

#중급반 2주차

분할 정복

<Divide and Conquer>

3p /<재귀 복습>

6p /<머지 소트>

12p /<빠른 거듭제곱>

21p /<연습 문제>

#분할 정복

<Divide and Conquer>

큰 문제를 작은 문제로 나누고,
작은 문제의 답으로 전체 답을 구하는 방법

- 1) 분할: 더 작은 같은 형태의 문제로 나눕니다.
- 2) 정복: 부분 문제를 재귀적으로 해결합니다.
- 3) 결합: 부분 문제의 답으로 현재 문제의 답을 만듭니다.

문제가 충분히 작아지면 재귀 호출 없이 직접 해결합니다.

#재귀

<Recursion>

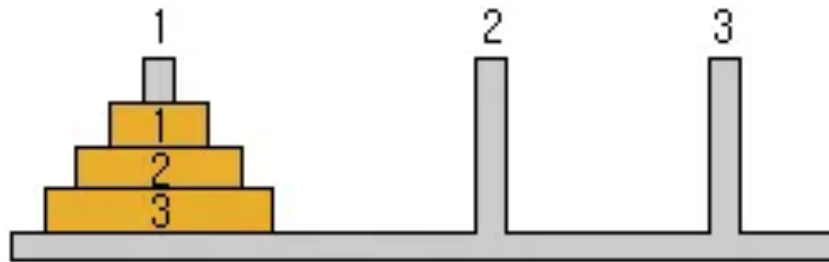
1 하노이1 (JUNGOL #1161)

<문제 설명>

- 1번 기둥에 있는 원판 N개를 3번 기둥으로 하나씩 옮길 때,
- 최소 횟수로 이동하는 과정을 순서대로 출력
- 단, 큰 원판이 작은 원판 위에 있을 수 없음

<제약 조건>

- $1 \leq N \leq 15$





#재귀

<Recursion>

1 하노이1 (JUNGOL #1161)

1) 분할: 더 작은 같은 형태의 문제로 나눕니다.

먼저, 1 ~ N-1 번 원판을 1번 기둥에서 2번 기둥으로 보내고

-> N번 원판을 1번 기둥에서 3번 기둥으로 보내고

-> 1 ~ N-1 번 원판을 2번 기둥에서 3번 기둥으로 보냅니다.

2) 정복: 부분 문제를 재귀적으로 해결합니다.

3) 결합: 부분 문제의 답으로 현재 문제의 답을 만듭니다.

#재귀

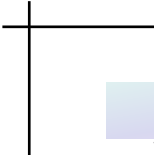
<Recursion>

1 하노이1 (JUNGOL #1161)

```
#include <iostream>
using namespace std;

void hanoi(int n, int from, int to, int via) {
    if (n == 0) return;
    hanoi(n - 1, from, via, to);
    cout << n << " : " << from << " -> " << to << '\n';
    hanoi(n - 1, via, to, from);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    int n;
    cin >> n;
    hanoi(n, 1, 3, 2);
}
```



#머지 소트

<Merge Sort>

배열을 반으로 나누어 정렬한 뒤,
정렬된 두 구간을 하나로 합칩니다.

- 1) 원소가 0개 또는 1개이면 이미 정렬되어 있습니다.
- 2) 왼쪽 절반과 오른쪽 절반을 각각 재귀적으로 정렬합니다.
- 3) 두 정렬된 구간을 작은 값부터 차례대로 합칩니다.

#머지 소트

<Merge Sort>

8개

7	2	6	3	8	1	5	4
---	---	---	---	---	---	---	---

4개씩

7	2	6	3	8	1	5	4
---	---	---	---	---	---	---	---

2개씩

7	2	6	3	8	1	5	4
---	---	---	---	---	---	---	---

1개씩

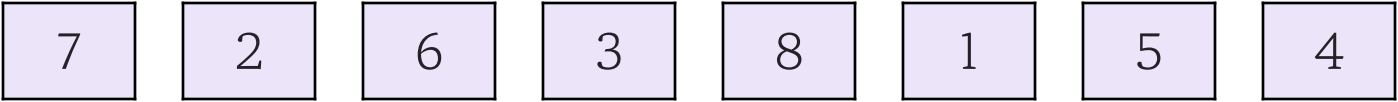
7	2	6	3	8	1	5	4
---	---	---	---	---	---	---	---

원소가 하나인 구간에 도달하면 재귀 호출을 멈춥니다.

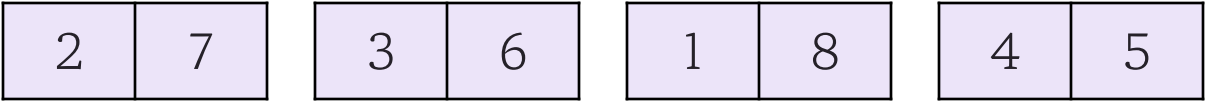
#머지 소트

<Merge Sort>

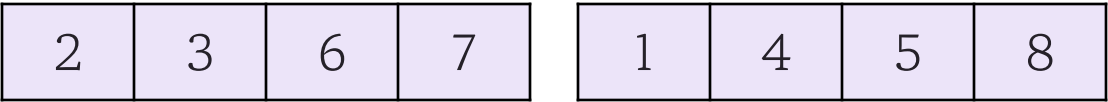
1개씩



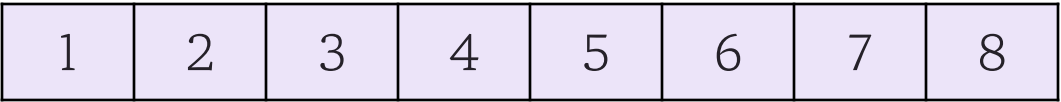
2개씩



4개씩



8개



각 단계에서 합치는 두 구간은 이미 정렬되어 있습니다.

#머지 소트

<Merge Sort>

i 와 j 는 각 구간에서 아직 사용하지 않은 첫 원소를 가리킵니다.

$a[i]$ 와 $a[j]$ 중 작은 값을 임시 배열 tmp에 넣습니다.

왼쪽 구간

2	3	6	7
---	---	---	---

 $i = 0$

오른쪽 구간

1	4	5	8
---	---	---	---

 $j = 4$

tmp

1							
---	--	--	--	--	--	--	--

지난 주에 배운 “투 포인터”를 응용해서, 간단하게 구현할 수 있습니다.

#머지 소트

<Merge Sort> - 시간 복잡도

단계	구간 개수	한 구간의 길이	단계 전체의 병합 비용
0	1	n	$O(n)$
1	2	$n / 2$	$O(n)$
2	4	$n / 4$	$O(n)$
...	...	...	...

$$O(n) \times O(\log n) = O(n \log n)$$

$$T(n) = 2T(n / 2) + O(n)$$

임시 배열 $O(n)$ 과 재귀 호출 깊이 $O(\log n)$ 을 곱해, 다음과 같은 시간복잡도를 얻습니다.

#머지 소트

<Merge Sort>

```
vector<int> tmp; // 병합할 때 쓰는 임시 배열

// 두 정렬된 구간 [l, m)과 [m, r)을 하나로 합친다
void merge(vector<int>& A, int l, int m, int r) {
    int i = l, j = m, k = l;

    // 두 구간의 앞부터 비교해 작은 값을 먼저 담는다
    while (i < m && j < r) {
        if (A[i] <= A[j]) tmp[k++] = A[i++];
        else tmp[k++] = A[j++];
    }

    // 남은 원소를 그대로 옮긴다
    while (i < m) tmp[k++] = A[i++];
    while (j < r) tmp[k++] = A[j++];

    // 합친 결과를 원래 배열에 복사한다
    for (int t = l; t < r; t++) A[t] = tmp[t];
}
```

```
// 구간 [l, r)을 병합 정렬한다
void mergeSort(vector<int>& A, int l, int r) {
    // 기저 조건: 원소가 1개 이하면 이미 정렬됨
    if (r - l <= 1) return;

    // 1. 분할: 구간을 반으로 나눈다
    int m = (l + r) / 2;

    // 2. 정복: 각 절반을 재귀적으로 정렬한다
    mergeSort(A, l, m);
    mergeSort(A, m, r);

    // 3. 병합: 정렬된 두 절반을 합친다
    merge(A, l, m, r);
}

int main() {
    // 입력
    int N;
    cin >> N;
    vector<int> A(N);
    for (int i = 0; i < N; i++) cin >> A[i];

    // 정렬
    tmp.resize(N);
    mergeSort(A, 0, N);

    // 출력
    for (int i = 0; i < N; i++) cout << A[i] << ' ';
    cout << '\n';
}
```



#거듭제곱

<Exponentiation>

1 제공수 출력 (JUNGOL #1092)

<문제 설명>

- X^Y 를 20091024 로 나눈 나머지를 출력하라.
- 편의상, 0^0 은 1으로 정한다.

<제약 조건>

- $0 \leq X, Y \leq 2,147,384,647$

#거듭제곱

<Exponentiation>

단순히 곱셈을 Y번 하면, 시간 복잡도는 $O(Y)$ 입니다.

```
long long result = 1;
for (long long i = 0; i < y; ++i) {
    result = result * x % MOD;
}
```

Y가 최대일 때 반복 횟수는 약 21억 번입니다.

이미 계산한 거듭제곱을 다시 쓸 수 있을까요?

#거듭제곱

<Exponentiation>

지수	계산 방법	예시
$y = 0$	1	$a^0 = 1$
$y = 2k$	$a^k \times a^k$	$a^{10} = a^5 \times a^5$
$y = 2k + 1$	$a^k \times a^k \times a$	$a^{11} = a^5 \times a^5 \times a$

a^k 을 한 번 구한 뒤, 그 값을 두 번 사용합니다.

매 호출에서 지수는 정수 나눗셈 $y / 2$ 로 줄어듭니다.

#거듭제곱

<Exponentiation>

3^{13} 을 계산해 볼까요?

지수: y	아래 호출에서 받은 값: X^k	이번 호출의 계산	반환값
13	729	$729 \times 729 \times 3$	1,594,323
6	27	27×27	729
3	3	$3 \times 3 \times 3$	27
1	1	$1 \times 1 \times 3$	3
0	기저 조건	1	1

호출할 때는 위에서 아래로, 값을 돌려줄 때는 아래에서 위로 읽습니다.

#거듭제곱

<Exponentiation>

동일한 재귀 호출을 두 번 실행하면, 분할 정복의 효과가 없습니다.

```
return power(a, y / 2) * power(a, y / 2);
```

$$T(y) = 2T(y / 2) + O(1) = O(y)$$

한 번 계산한 값을 저장해서 재사용하면 문제를 효율적으로 풀 수 있습니다.

```
long long half = power(a, y / 2);  
return half * half;
```

$$T(y) = T(y / 2) + O(1) = O(\log y)$$

#모듈러 연산

<Modulo

$$(a \times b) \bmod M = ((a \bmod M) \times (b \bmod M)) \bmod M$$

곱셈을 할 때마다 나머지를 취해도 최종 나머지는 같습니다.

입력 X를 먼저 M으로 나눈 나머지로 바꿉니다.

곱셈은 long long으로 계산합니다.

```
long long result = half * half % MOD;  
if (y % 2 == 1) result = result * a % MOD;
```

M = 20,091,024에서는 두 나머지의 곱이 64비트 정수 범위 안에 들어갑니다.

#거듭제곱

<Exponentiation>

1 제곱수 출력 (JUNGOL #1092)

```

#include <iostream>
using namespace std;

const long long MOD = 20091024;

// a^y % MOD 를 분할 정복으로 구한다
long long power(long long a, long long y) {
    // y = 0 : a^0 = 1 (0^0도 1로 정함)
    if (y == 0) return 1;

    // a^(y/2)를 한 번만 구해서 저장한다
    // (power(a, y/2)를 두 번 호출하면 0(y)가 되어 분할 정복의 효과가 없다)
    long long half = power(a, y / 2);

    // y = 2k : a^k × a^k
    // 곱할 때마다 나머지를 취해도 최종 나머지는 같다
    long long result = half * half % MOD;

    // y = 2k + 1 : a^k × a^k × a
    if (y % 2 == 1) result = result * a % MOD;

    return result;
}

int main() {
    long long X, Y;
    cin >> X >> Y;

    // 입력 X를 먼저 MOD로 나눈 나머지로 바꾼다
    // 두 나머지의 곱이 64비트 정수(long long) 범위 안에 들어간다
    cout << power(X % MOD, Y) << '\n';
}

```

#거듭제곱

<Exponentiation>

1 제곱수 출력 (JUNGOL #1092)

```
#include <iostream>
using namespace std;

typedef long long ll;

// a^b (mod c) 를 구하는 함수 (반복문을 이용한 빠른 거듭제곱)
// 시간 복잡도: O(log b)
ll PowMod(ll a, ll b, ll c) {
    // 어떤 수든 1로 나눈 나머지는 0
    if (c == 1) return 0;

    ll res = 1; // 결과값 (곱셈의 항등원 1로 시작)

    // a %= c : 시작 전에 a를 c로 나눈 나머지로 줄인다
    // b : 지수가 0이 될 때까지 반복
    // b >>= 1 : 지수를 오른쪽으로 한 비트 민다 (b /= 2)
    // a = a*a%c: 밑을 제공한다 (a → a^2 → a^4 → a^8 ...)
    for (a %= c; b; b >>= 1, a = a * a % c)
        // b의 현재 최하위 비트가 1이면 지금의 a를 결과에 곱한다
        if (b & 1) res = res * a % c;

    return res;
}
```

#마스터 정리

<Master Theorem>

Combining the three cases above gives us the following “master theorem”.

Theorem 1 *The recurrence*

$$T(n) = aT(n/b) + cn^k$$

$$T(1) = c,$$

where a , b , c , and k are all constants, solves to:

$$T(n) \in \Theta(n^k) \text{ if } a < b^k$$

$$T(n) \in \Theta(n^k \log n) \text{ if } a = b^k$$

$$T(n) \in \Theta(n^{\log_b a}) \text{ if } a > b^k$$

<https://math.stackexchange.com/q/646908>

#예제

1 하노이1 (JUNGOL #1161)

회합에서 다룬 재귀함수 복습 문제

1 제곱수 출력 (JUNGOL #1092)

회합에서 다룬 분할 정복 문제

#연습 문제

4 Tutorial : 합병(병합)정렬(Merge Sort) (JUNGOL #3519)

머지 소트에 대한 더 자세한 설명과 직접 구현해볼 수 있는 예제

2 색종이 만들기(영역구분) (JUNGOL #1335)

재귀함수 연습 문제

5 색종이 만들기 3(압축해제) (JUNGOL #3249)

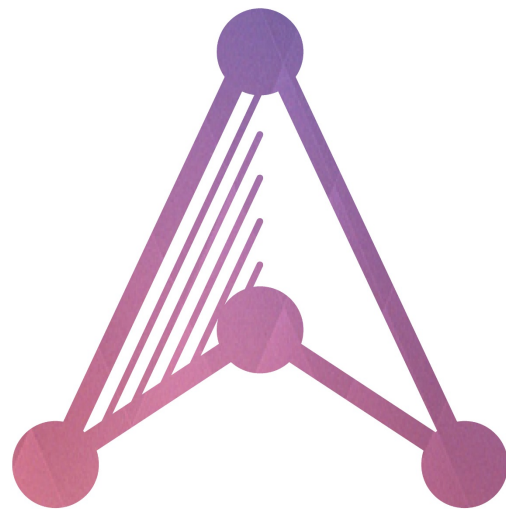
조금은 구현이 어려운 분할 정복

2 피보나치 (JUNGOL #1053)

행렬을 이용해 볼까요?

5 타일 채우기 (JUNGOL #2543)

UCPC 2021 예선 I번(P2)의 쉬운 버전



A L O H A
The algorithm club.